Project Title: International Cargo & Customs Clearance Management System

Business Domain: *Freight Forwarding & International Logistics*

1. Overview

The International Cargo & Customs Clearance Management System manages cross-border freight workflows end to end, including shipment booking, container loading, customs documentation, cargo tracking, and freight invoicing. Built using an MVC architecture, it supports both Java (Spring MVC) and .NET (ASP.NET Core MVC) frameworks.

The system consists of five key modules:

1. **Shipment Booking & Bill of Lading Generation**
2. **Container Loading & Vessel Manifest Management**
3. **Customs Documentation & Duty Calculation**
4. **Cargo Tracking & Port Operations**
5. **Freight Invoicing & Settlement**

2. Assumptions

- Database: MySQL or SQL Server.
- Roles: Freight Forwarder, Customs Broker, Port Operator, Shipper, Consignee, Admin.
- ORM: Hibernate (Java), Entity Framework Core (.NET).
- Shipping line and port authority interfaces are simulated through mock APIs.
- Customs tariff and HS code lookups use mock services.

3. Module-Level Design

3.1 Shipment Booking & Bill of Lading Generation Module

Purpose: Captures shipment bookings and generates Bills of Lading.

Controller:

- ShipmentBookingController
  - createBooking()
  - generateBillOfLading()
  - amendBooking()
  - getBookingDetails()

Service:

- ShipmentBookingService
  - Validates shipper/consignee details, allocates equipment, and issues B/L numbers.

Model:

- ShipmentBooking Entity
  - Attributes:
    - bookingId (PK)
    - bookingNumber
    - shipperName
    - consigneeName
    - originPort
    - destinationPort
    - bookingStatus (DRAFT, CONFIRMED, CANCELLED, COMPLETED)

3.2 Container Loading & Vessel Manifest Management Module

Purpose: Manages container packing and builds vessel manifests.

Controller:

- ContainerController
  - assignContainer()
  - loadContainer()
  - generateVesselManifest()
  - getContainerDetails()

Service:

- ContainerService
  - Tracks container types, seal numbers, and stowage plans for each voyage.

Model:

- Container Entity
  - Attributes:
    - containerId (PK)

                - containerNumber
                - containerType (DRY, REEFER, FLAT_RACK, OPEN_TOP)
                - bookingId (FK)
                - sealNumber
                - containerStatus (EMPTY, LOADED, IN_TRANSIT, DISCHARGED)

3.3 Customs Documentation & Duty Calculation Module

Purpose: Prepares customs declarations and calculates import/export duties.

Controller:

- CustomsController
    - submitCustomsDeclaration()
    - calculateDuty()
    - uploadCustomsDocument()
    - getClearanceStatus()

Service:

- CustomsService
    - Applies HS codes, tariff rates, and validates required documentation.

Model:

- CustomsDeclaration Entity
    - Attributes:
        - declarationId (PK)
        - bookingId (FK)
        - hsCode
        - declaredValue
        - calculatedDuty
        - clearanceStatus (FILED, UNDER_REVIEW, CLEARED, HELD, REJECTED)

3.4 Cargo Tracking & Port Operations Module

Purpose: Tracks cargo movement across ports and milestones.

Controller:

- TrackingController
    - recordPortEvent()
    - updateLocation()
    - getCargoMilestones()
    - logDelay()

Service:

- TrackingService
    - Records gate-in, loaded, sailed, discharged, and gate-out events.

Model:

- CargoEvent Entity
    - Attributes:
        - eventId (PK)
        - containerId (FK)
        - eventType (GATE_IN, LOADED, SAILED, DISCHARGED, GATE_OUT)
        - eventLocation
        - eventTimestamp
        - remarks

3.5 Freight Invoicing & Settlement Module

Purpose: Generates freight invoices and tracks settlement.

Controller:

- FreightInvoiceController
    - generateFreightInvoice()
    - applyDemurrage()
    - recordPayment()
    - getInvoiceLedger()

Service:

- FreightInvoiceService
    - Computes ocean freight, surcharges, demurrage, and detention charges.

Model:

- FreightInvoice Entity
  - Attributes:
    - invoiceId (PK)
    - bookingId (FK)
    - freightCharges
    - surchargeAmount
    - totalAmount
    - currency
    - invoiceStatus (DRAFT, ISSUED, PAID, OVERDUE)

4. Database Schema

ShipmentBooking Table

```
CREATE TABLE ShipmentBooking (
bookingId INT AUTO_INCREMENT PRIMARY KEY,
bookingNumber VARCHAR(30),
shipperName VARCHAR(100),
consigneeName VARCHAR(100),
originPort VARCHAR(50),
destinationPort VARCHAR(50),
bookingStatus ENUM('DRAFT','CONFIRMED','CANCELLED','COMPLETED')
);
```

Container Table

```
CREATE TABLE Container (
containerId INT AUTO_INCREMENT PRIMARY KEY,
containerNumber VARCHAR(20),
containerType ENUM('DRY','REEFER','FLAT_RACK','OPEN_TOP'),
bookingId INT,
sealNumber VARCHAR(30),
containerStatus ENUM('EMPTY','LOADED','IN_TRANSIT','DISCHARGED'),
FOREIGN KEY (bookingId) REFERENCES ShipmentBooking(bookingId)
);
```

CustomsDeclaration Table

```
CREATE TABLE CustomsDeclaration (
declarationId INT AUTO_INCREMENT PRIMARY KEY,
bookingId INT,
hsCode VARCHAR(20),
declaredValue DECIMAL(15,2),
calculatedDuty DECIMAL(15,2),
clearanceStatus ENUM('FILED','UNDER_REVIEW','CLEARED','HELD','REJECTED'),
FOREIGN KEY (bookingId) REFERENCES ShipmentBooking(bookingId)
```

```
);
```

CargoEvent Table

```
CREATE TABLE CargoEvent (
eventId INT AUTO_INCREMENT PRIMARY KEY,
containerId INT,
eventType ENUM('GATE_IN','LOADED','SAILED','DISCHARGED','GATE_OUT'),
eventLocation VARCHAR(100),
eventTimestamp DATETIME,
remarks VARCHAR(255),
FOREIGN KEY (containerId) REFERENCES Container(containerId)
);
```

FreightInvoice Table

```
CREATE TABLE FreightInvoice (
invoiceId INT AUTO_INCREMENT PRIMARY KEY,
bookingId INT,
freightCharges DECIMAL(15,2),
surchargeAmount DECIMAL(15,2),
totalAmount DECIMAL(15,2),
currency VARCHAR(10),
invoiceStatus ENUM('DRAFT','ISSUED','PAID','OVERDUE'),
FOREIGN KEY (bookingId) REFERENCES ShipmentBooking(bookingId)
);
```

5. Local Deployment Details

- Install DB + JDK 17/21/24 or .NET SDK 8.0
- Deploy using Tomcat or Kestrel

Steps:

1. Clone repository.
2. Run SQL schema.
3. Configure app settings files.
4. Build & run the project.

6. Conclusion

The International Cargo & Customs Clearance Management System provides a robust MVC-based structure for global freight workflows, customs compliance, and cross-border logistics. It is ideal for training scenarios and local deployment.